

Running microPASCAL80 on 6809/6800 and FLEX: SBC8080

 flexonsbd.blogspot.com/2019/05/sbc8080mropascal80.html

Tuesday, May 14, 2019



Running microPASCAL80 on SBC8080

I ran microPASCAL80 on an SBC8080 (CPU board + SUB board).

In my blog post on April 27th, I reported that GAME80 was running on the SBC8080. [Skyriver](#), [who](#) saw my post, managed to run not only the GAME80 interpreter but also the compiler on CP/M80. Using that as a reference, I was able to run the compiler on my SBC8080 as well.

I used GAMEIII on a 68-series computer, and while the ease with which you can manipulate the computer is very appealing compared to assembly language, am I the only one who thinks the program readability is somewhat lacking? (Or maybe I just couldn't get used to it?) Back then, I longed for UCSD PASCAL.

About a year and a half after GAMEIII, when Sadanobu Maruyama announced microPASCAL80 in ASCII magazine (January 1980 issue), I, a 68-series user, could only watch enviously.

Finally, in the November issue, Michiaki Kawasaki announced microPASCAL68, and of

course, I worked hard to input a binary of nearly 5KB. It still runs on my homemade 6802 board.

This time, I made an SBC8080 board and got GAME80 to work, so I decided to try running microPASCAL80, which I had long admired back then.

Mr. Maruyama's presentation included the source code, so I thought it would be easy, including changing the addresses, but I was troubled by the characters of the discharge printers of that time.

It did run, but I was troubled by a strange situation where I couldn't assign variables, but after reviewing it many times, I finally found a single character input error in the source code, corrected it, and it worked successfully.

So, I put the GAME80 interpreter, GAME80 compiler, and microPASCAL80 into the ROM on the SBC8080SUB board and finished it by selecting and executing them with a custom Load command from the monitor.

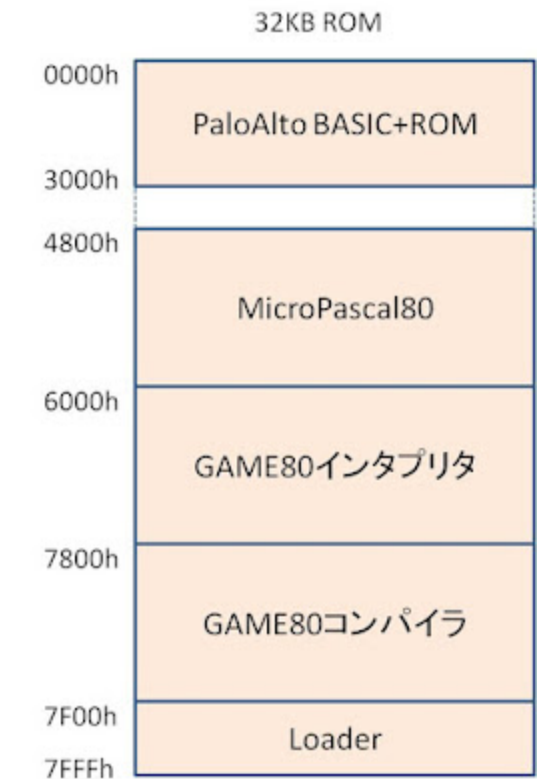
(1) GAME80 interpreter Since the original is binary and not source code, I converted the binary file starting from address 8600h into a HEX file so that it fits into the ROM starting from 7800h. → GAME80-7800.hex

(2) GAME80 Compiler Since the original is a GAME80 program, it would be fine to load it while the GAME80 interpreter is running, but to save that trouble, it is converted to a memory storage format in advance and stored in ROM, and then transferred to memory starting from A000h. It is converted to a HEX file so that it will be stored starting from 6000h in ROM. For conversion to the storage format, we are using ~~CvtMotHexBin.exe~~ which was published in the previous blog. → GAME80C-6000.hex

[Correction on May 25] The program for conversion is ChgIno.exe , not CvtMotHexBin.exe .

(3) microPASCAL80 Interpreter Since the original is source, the starting address is changed from the original 2000h to 8000h and assembled. The resulting binary is converted to a HEX file so that it will be stored starting from 4800h in ROM. → MPas80-4800.hex

(4) Loader The source code starts from 7F00h, so we assemble it as is and convert it to a HEX file. → LOADER.hex (For conversion to save format and to HEX format, we use our custom ChgIno.exe and CvtMotHexBin.exe, which were published in the previous blog .)



Layout diagram in ROM

[Correction May 16] The positions of the GAME80 interpreter and GAME80 compiler in the diagram above are swapped. The correct positions are compiler from 6000h and interpreter from 7800h.

These four HEX files are added to the end of the PTBEXSA.HEX file in the SBC8080's datapck to create the ROM file.

A summary of the changes in each file is provided below.

- (1) GAME80 As written in "GAME80 finally works on SBC8080" published on April 27.
- (2) GAME80C Changes and modifications as described in [skyriver](#)'s blog are made.
- (3) MPas80 The changes and additions are shown below.

Original

```

2792 32D9 CDC003 BREAK: CALL PLIN
2793 32DC FE03 CPI 03H
2794 32DE C0 RNZ

```

```

2795 32DF C30320 JMP START2
2796 ;
2797 32E2 CDB903 GETCH: CALL GETCHR
2798 32E5 CAE232 JZ GETCH
2799 32E8 C3DC32 JMP BREAK+3 changes/additions 2793 92D9 CDFA92 BREAK: CALL
PLIN 2794 92DC FE03 CPI 03H 2795 92DE C0 RNZ 2796 92DF C30380 JMP START2
2797; 2798 92E2 CDF792 GETCH: CALL GETCHR 2799 ; JZ GETCH 2800 92E5 00 NOP
2801 92E6 00 NOP 2802 92E7 00 NOP 2803 92E8 C3DC92 JMP BREAK+3 2804
2805 ;+++++ 2806 ;* I/O ROUTINE 2807
2808 92EB F5 PUTCHR: PUSH PSW 2809 92EC DB01 PUT0 IN 01H 2810 92EE
E601 ANI 01H 2811 92F0 CAEC92 JZ PUT0 2812 92F3 F1 POP PSW 2813 92F4 D300
OUT 00H 2814 92F6 C9 RET 2815 2816 92F7 C37C00 GETCHR: JMP 007CH
2817 2818 92FA 3E00 PLIN: MVI A,00H 2819 92FC C9 RET

```

(4) Loader The source code is also included in the files that will be released. (The transfer routine was borrowed from the explanation of the 8080 in "My Own Microcomputer Design & Construction" by Yoshihiko Matsumoto.)

In all of (1), (2), and (4), the input routine uses the routines in the monitor.

The image below shows the startup and operation. The program entered as an example is the factorial calculation program from the article on microPASCAL68.



```
COM3:9600bps - Tera Term VT
ファイル(F) 編集(E) 設定(S) コントロール(O) ウィンドウ(W) ヘルプ(H)

PALO ALTO TINY BASIC
MON80 EXTENDED
SBC8080 EDITION

OK
>MON
[8000]EXEC 7F00

(1:GAME8080C, 2:GAME80C, 3:MPASCAL80)[1-3]?
[8000]EXEC 8000

*** MICRO PASCAL *** VER 2.2

READY.
>B
% KAIJO %
VAR I;
FUNC FACT(I);
  IF I=0 THEN FACT:=1
  ELSE FACT:=I*FACT(I-1);
% MAIN %
FOR I:=0 TO 7 DO
  WRITE(I,"I=",FACT(I):1,./).
>G/
0I=1
1I=1
2I=2
3I=6
4I=24
5I=120
6I=720
7I=5040

READY.
>
```

Startup/Operation Screen

Now, just like with the 68 board, I can use two languages, GAME and microPASCAL, so it seems I can play around with the SBC8080 without touching the 8080 machine code.

- (1) GAME80 binary and HEX file
 - (2) GAME80C converted to memory storage format and its HEX file
 - (3) MPas80 binary and its HEX file
 - (4) Loader source and binary HEX file I've uploaded
- all of these together to [OneDrive](#)

. Nowadays, anyone can use high-level languages as a matter of course, so I don't think GAME or MicroPascal will seem all that appealing, but when I think back to nearly 40 years ago, when these languages were running on incredibly functionally poor computers, I'm filled with emotion.